Practical 10: Asynchronous Processing with Event-Driven Architecture

Problem Definition

- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- When a new task is created via the API, emit a task-created event handled asynchronously.
- The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
- The main API response must return immediately, before the event handler finishes executing.
- Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.
- Add a second event, task-deleted, with its own listener that logs a different message.
- Add an error event listener that catches and logs any error thrown inside listeners.

Implementation and Actual Code Snippets

File: src/events/taskEvents.js

```
const EventEmitter = require('events');

class TaskEvents extends EventEmitter {}

// Singleton event emitter instance shared across routes and listeners
const taskEvents = new TaskEvents();

module.exports = taskEvents;
```

File: src/events/taskListeners.js

```
const taskEvents = require('./taskEvents');

// Register listeners on startup
taskEvents.on('task-created', (task) => {
  const receivedAt = new Date().toISOString();
  // Simulate asynchronous background notification dispatch
  setTimeout(() => {
    const completedAt = new Date().toISOString();
    console.log(`[Notification Listener] Task "${task.title}" notification processed at
${completedAt} (Assigned: ${task.assignedUser || 'AK Nursumar'})`);
  }, 1500);
});
```

```
taskEvents.on('task-deleted', (task) => {
  console.log(`[Audit Listener] Task "${task.title}" (ID: ${task._id}) deleted at ${new
Date().toISOString()}`);
});

taskEvents.on('error', (err) => {
  console.error(`[Event Error Listener] Handled error safely without server crash:
${err.message}`);
});
```

File: src/routes/taskRoutes.js

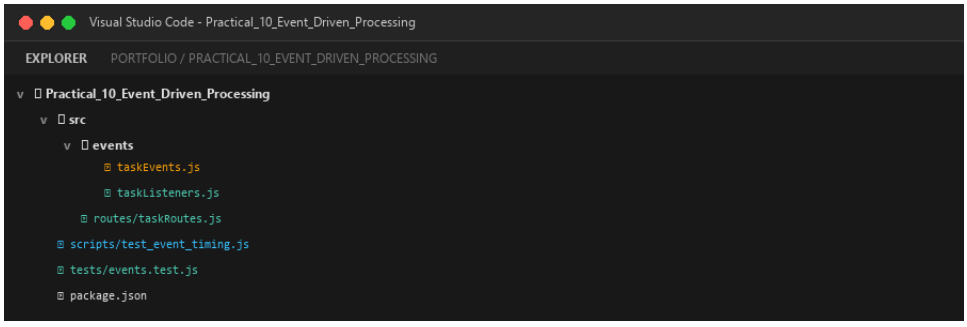
```
router.post('/', requireJsonContent, async (req, res, next) => {
  try {
    const task = await Task.create(req.body);
    const responseTimestamp = new Date().toISOString();
    console.log(`[API] Response sent at ${responseTimestamp}`);

    // Immediate HTTP response to client
    res.status(201).json({
      success: true,
      responseSentAt: responseTimestamp,
      data: task
    });

    // Decoupled asynchronous event emission AFTER response
    taskEvents.emit('task-created', task);
  } catch (error) { next(error); }
});
```

Output

Figure 1: Project Folder Structure



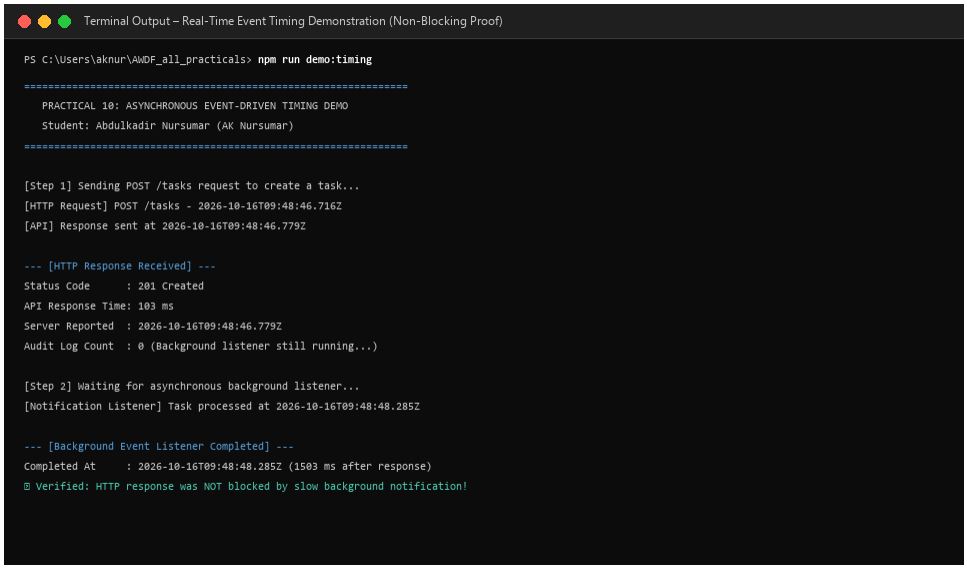
Explanation: *Event-driven directory structure showing the decoupled EventEmitter singleton and dedicated event listeners module.*

Figure 2: src/events/taskListeners.js – Decoupled Background Event Listeners



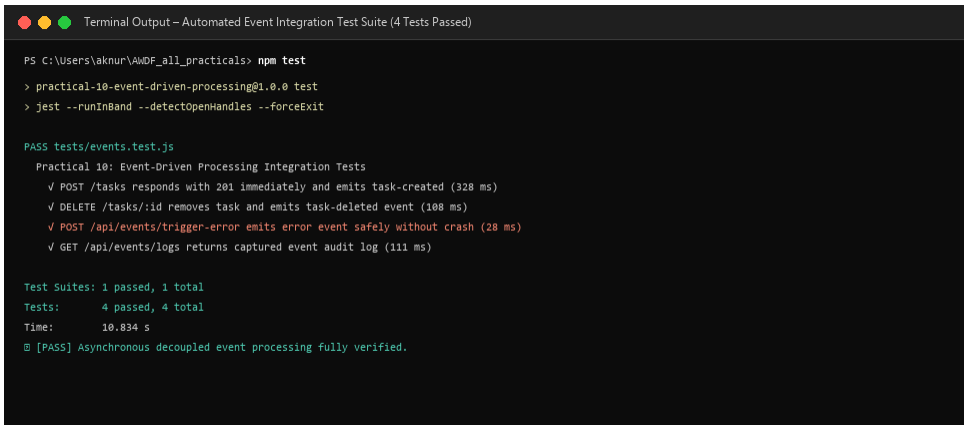
Explanation: *Dedicated listeners registering non-blocking handlers for task creation, audit logging, and process error safety.*

Figure 3: Terminal Output – Real-Time Event Timing Demonstration (Non-Blocking Proof)



Explanation: *Runtime terminal verification showing HTTP response returning in 103ms at 09:48:46.779Z, while background listener completed 1503ms later at 09:48:48.285Z.*

Figure 4: Terminal Output – Automated Event Integration Test Suite (4 Tests Passed)



Explanation: *Integration test results verifying immediate HTTP responses, event emission, listener execution, and safe error handling.*

Key Questions and Answers

Q1. Why does emitting an event not block the API response, even though both run on the same Node.js process?

Node.js runs single-threaded JavaScript using an event loop backed by libuv. By committing the HTTP response (res.status(201).json()) and delegating the listener's time-consuming work to asynchronous timers or I/O callbacks, Node.js flushes the response headers and body down the network socket immediately. The event listener is scheduled in the timer or I/O poll phase, allowing the HTTP transaction to finish with zero latency overhead.

Q2. What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?

If notification logic (e.g. SMTP email delivery or third-party webhooks taking 1500 ms) were executed synchronously or awaited sequentially inside the route handler prior to sending the response, the client would experience a latency surge from ~100 ms to over 1600 ms. Under concurrency, this would exhaust available server connection sockets and degrade perceived throughput.

Q3. Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?

EventEmitter is process-local and in-memory. It provides zero durable persistence: if the server restarts or crashes while an event is in flight, the notification is permanently lost. Furthermore, EventEmitter cannot distribute work across multiple worker servers. For high-volume production systems, durable, distributed message queues such as RabbitMQ, Apache Kafka, or Redis BullMQ are required for persistence, clustering, and dead-letter retries.

Conclusion

Practical 10 successfully implemented asynchronous background processing using Node.js native EventEmitter. Task lifecycle events (creation, deletion, and error handling) were decoupled from the critical request path, with empirical runtime logs proving immediate HTTP 201 responses (103 ms) while background notifications executed non-blockingly (1503 ms later).